

Operating System Components as Verified Libraries

Annual report for VeTSS grant, April 2018 to March 2019

Author: Tom Ridge, University of Leicester

NOTE: this report intentionally follows the format of the previous VeTSS annual report (one page summary, with publications and other related information).

ImpFS | The next 700 filesystems

Abstract

ImpFS is a new **library filesystem**: a filesystem built out of many independent components. It is intended to be used in unikernels such as **MirageOS**, but it is also usable on desktops or in the cloud.

What makes ImpFS so great?

- Small components, each with a simple and formal semantics
- Strong separation of concerns; for example, everything is parameterized and as abstract as possible
- Fast! Writing data to disk is a slow operation. The combination of the B-tree with the persistent cache means that we achieve minimal and precise flushing of data to disk.
- Verification of components possible, e.g. previous versions of the B-tree have been verified in Isabelle
- Each component exhaustively tested.

Logos: EPSRC, REMS, VeTSS, UNIVERSITY OF LEICESTER, Microsoft

The Design of ImpFS

```
graph TD
    POSIX[POSIX filesystem API] --> TL[Translation layer]
    TL --> FDOA[File+directory object API]
    FDOA -- "File and dir operations" --> Layers[...further layers...]
    FDOA -- "Sync'd updates to object roots" --> GOM[Global object map]
    GOM --> PKVS[Persistent key value store]
    PKVS --> LRU[LRU cache]
    LRU --> PC[Persistent cache]
    PC --> BT[B-tree]
    BT --> BD[Block device]
    Index[Index] --> GOM
    Index --> BT
```

POSIX filesystem API: Standard filesystem API, including sync commands, exposed via FUSE or over the network.

Translation layer: Translates POSIX API calls into operations on file and directory objects.

File and directory objects: Implementations of files and directories. Both use an underlying copy-on-write B-tree.

Global object map (GOM): Updates to objects that are made persistent on disk result in new "object root blocks". The map from object id to object root block is maintained in an index: the global object map.

Persistent key value store: The core of the GOM is implemented using an on-disk key-value store.

LRU cache: Updates to the key value store are persisted only when necessary. The LRU is an in-memory cache of operations that are flushed to disk on sync operations (or when the LRU is full).

Persistent cache: The implementation of the KV store uses a B-tree. Rather than update the B-tree directly we maintain an on-disk log of operations. At intervals, these are committed as a batch to the B-tree.

B-tree: The B-tree is a copy-on-write on-disk map. The persistent state of the GOM is a combination of the B-tree state and the persistent cache.

Block device: Where data is finally stored.

ImpFS is a project by Tom Ridge at the University of Leicester. google: tom ridge leicester

Introduction (bullet points)

- Core operating system functionality has been verified in projects such as L4.verified, which verified the seL4 microkernel.
- Two major components, the network stack and the file system, should also be verified.
- Although a few file systems have been verified, their performance is very slow compared to traditional file systems such as ext4 and ZFS. Moreover, they lack important features of modern file systems, such as file system snapshots.
- We aim to develop a verified file system "ImpFS" constructed from small, well-defined components. We aim to match or exceed existing file systems, both in terms of performance and features. The file system will be available both as a desktop file system, and as a library of components suitable for use in other software, and even in library-based unikernels such as MirageOS.

Main text

In previous work we developed SibylFS, a formal model of POSIX and real-world file systems. The formal specification was usable directly as a test oracle, to check conformance of real-world file systems. Now we wish to actually implement a verified file system.

Building a file system is hard. Building a high-performance file system with advanced features is extremely difficult: the BTRFS file system has been in development by a team of engineers for around 10 years and is still not considered stable enough for production use. Thus, we should expect that building a *verified* high-performance file system will be extremely challenging.

So far, we have developed many file system components. The most important components are:

- A high-performance novel B-tree-like datastructure with both Copy-on-Write and Mutate-in-place semantics. This is formalized in Isabelle/HOL and extracted to OCaml for execution.
- A persistent cache, offering transactional-log-like functionality.
- A persistent key-value store.

These components are freely available online from <http://www.tom-ridge.com/filesystems.html>. All the components are implemented in a purely-functional style, which is a pre-requisite for easy verification. The performance of the components is extremely good. For example, the key-value store matches the performance of the well-regarded LMDB key-value store.

Recently we have finalized the design for our ImpFS file system, and are in the process of implementing the design, prior to benchmarking. We are confident that the file system will out-perform existing file systems in many areas. More importantly, the components will serve as the basis for many further interesting file system designs.

The VeTSS funding has been critical for securing research time for the author. Most of the work has involved high-level design and low-level component implementation. In addition, a lot of effort has been put into performance engineering of the components. This work is lengthy and would not have been possible without the financial support provided by VeTSS.

Publications

A B-tree library for OCaml. Tom Ridge. ICFP 2017, OCaml'17 workshop.

Towards verified file systems. Andrea Giugliano, 2018. PhD thesis.

A Key-Value store for OCaml. Tom Ridge. ICFP 2019, OCaml'19 workshop.